

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

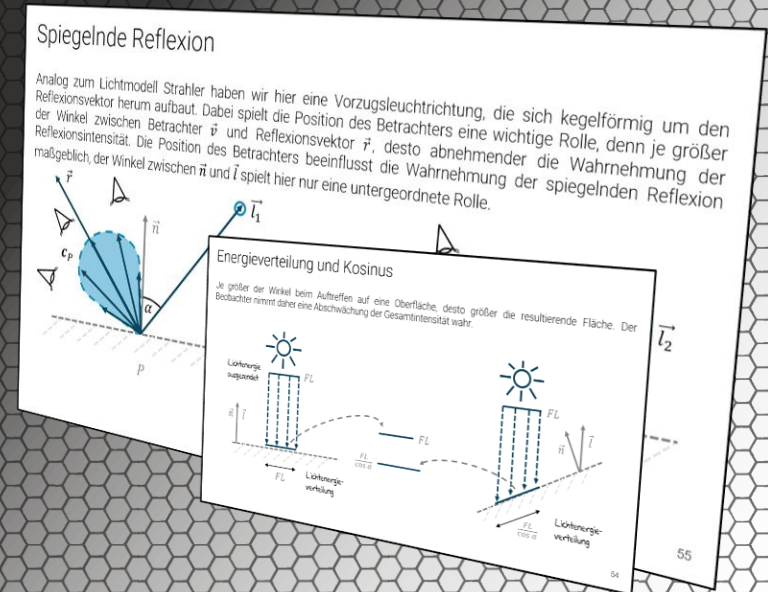
Sommersemester 2026



Elementare Beleuchtungsmodelle

„Ohne Licht gibt es keine Kunst.“ (Rembrandt van Rijn zugeschrieben)

PD Prof. Dr. Marco Block-Berlitz



Ziele und Inhalt

- Vereinfachtes Texture-Mapping
- Ambientes Licht
- Shader: Ambientes Licht
- Beispiel ambientes Licht
- Diffuse Reflexion
- Energieverteilung und Kosinus
- Diffuse Reflexion mit Richtungs- und Positionslicht
- Spiegelnde Reflexion
- Reflexionsvektor bestimmen
- Beschleunigung durch Halfway-Vektor
- Beschleunigung durch Fallunterscheidung
- Beschleunigung durch Exponentialalternative
- Emissionslicht
- Phong-Beleuchtungsmodell
- Shader: Erweitertes Phong-Beleuchtungsmodell
- OpenGL-Beleuchtungsmodell
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



Vereinfachtes Texture-Mapping

Für die folgenden Verfahren gehen wir zunächst von einem **vereinfachten Texture-Mapping** aus und erhalten für jedes einzufärbende Pixel genau eine Texturfarbe.



$$\text{3D-Mesh} + \text{Textur} = \text{Texturiertes 3D-Mesh}$$

Wenn wir an einer konkreten Position P sind, gehen wir der Einfachheit halber davon aus, dass dafür ein konkreter Texturfarbwert vorliegt:

$$\mathbf{c}_P = \mathbf{t}_P$$

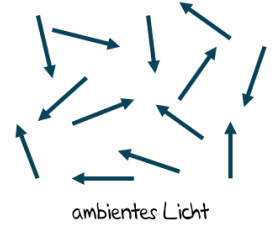


Elementares Beleuchtungsmodell

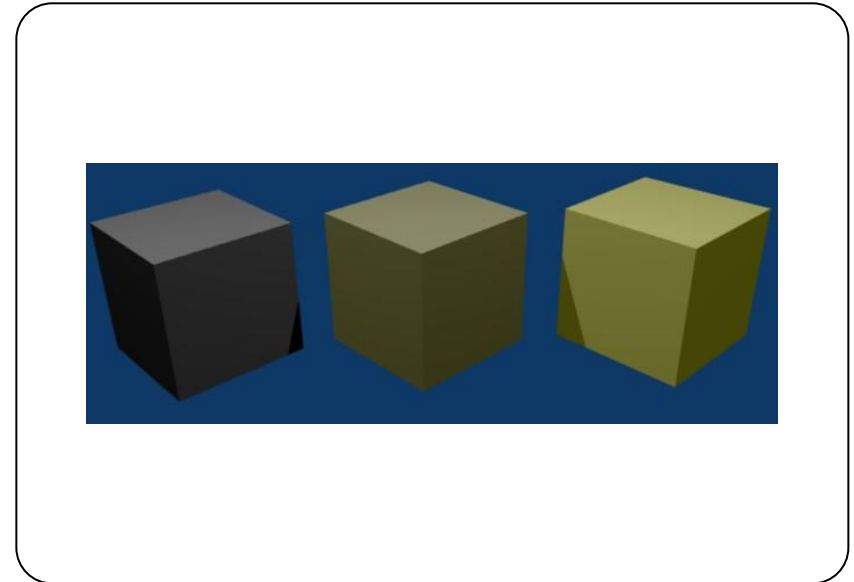
Ambientes Licht

Ambientes Licht

Wie bei den anderen Lichtquellen gibt es auch hier zunächst eine Farbe. **Ambientes Licht** (engl. *ambient light*), auch Streulicht genannt, hat keine Position und keine Richtung.



$$\mathbf{c}_P = \mathbf{c}_P^{(a)} \cdot \mathbf{c}_L$$



Wie reagiert das Material auf das ambiente Licht?

ambientes Licht

$$\mathbf{c}_P = \mathbf{a}_P \cdot \mathbf{a}$$



Shader: Ambientes Licht

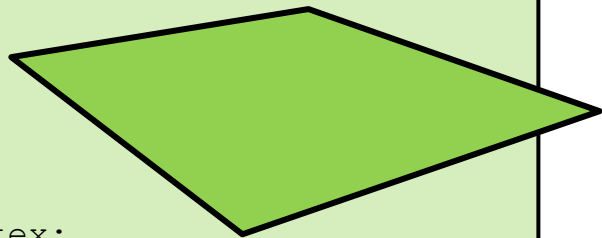


Vertex-Shader

```
varying vec3 color;

vec3 ambientLight    = vec3(1, 1, 1);
vec3 ambientMaterial = vec3(.2, .2, .2);

void main() {
    color          = ambientMaterial * ambientLight;
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



Shader: Ambientes Licht

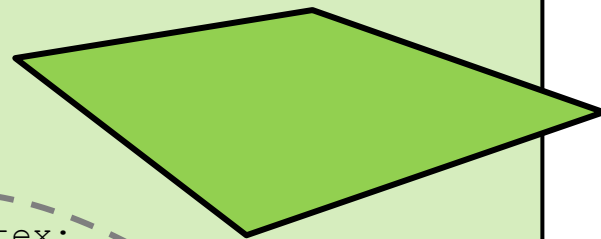


Vertex-Shader

```
varying vec3 color;

vec3 ambientLight    = vec3(1, 1, 1);
vec3 ambientMaterial = vec3(.2, .2, .2);

void main() {
    color          = ambientMaterial * ambientLight;
    gl_Position    = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



Fragment-Shader

```
varying vec3 color;

void main() {
    gl_FragColor = vec4(color, 1);
}
```

$$\mathbf{c}_P = \mathbf{a}_P \cdot \mathbf{a} = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.2 \end{bmatrix}$$

Beispiel ambientes Licht

Wenn wir eine Szene aufhellen wollen, dann führt der Einsatz von ambientem Licht dazu, Schattenbereiche heller erscheinen zu lassen, ohne mehr Struktur zu zeigen.



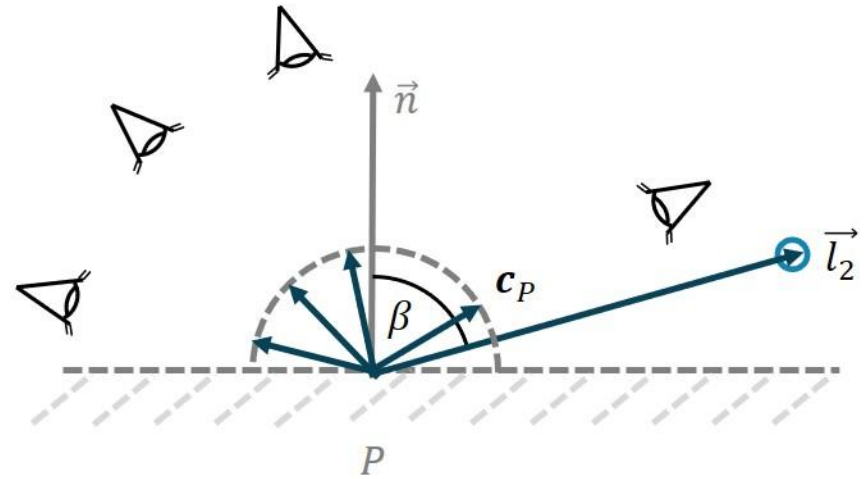
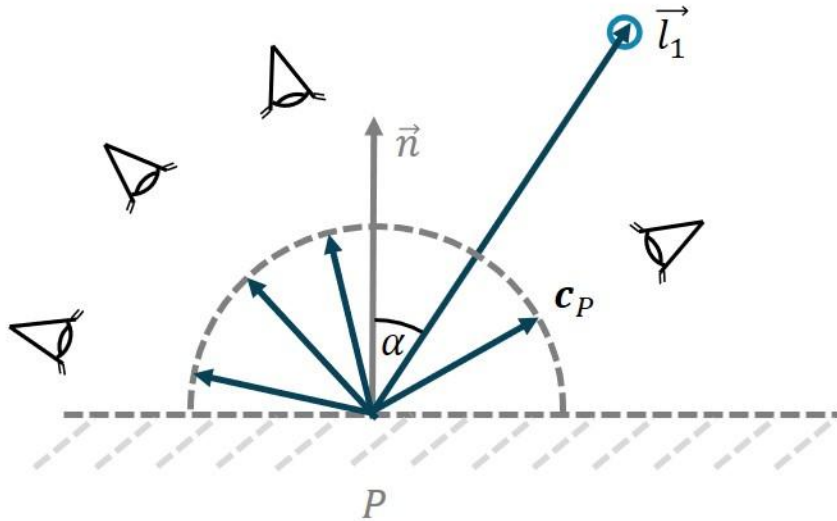


Elementares Beleuchtungsmodell

Diffuses Licht

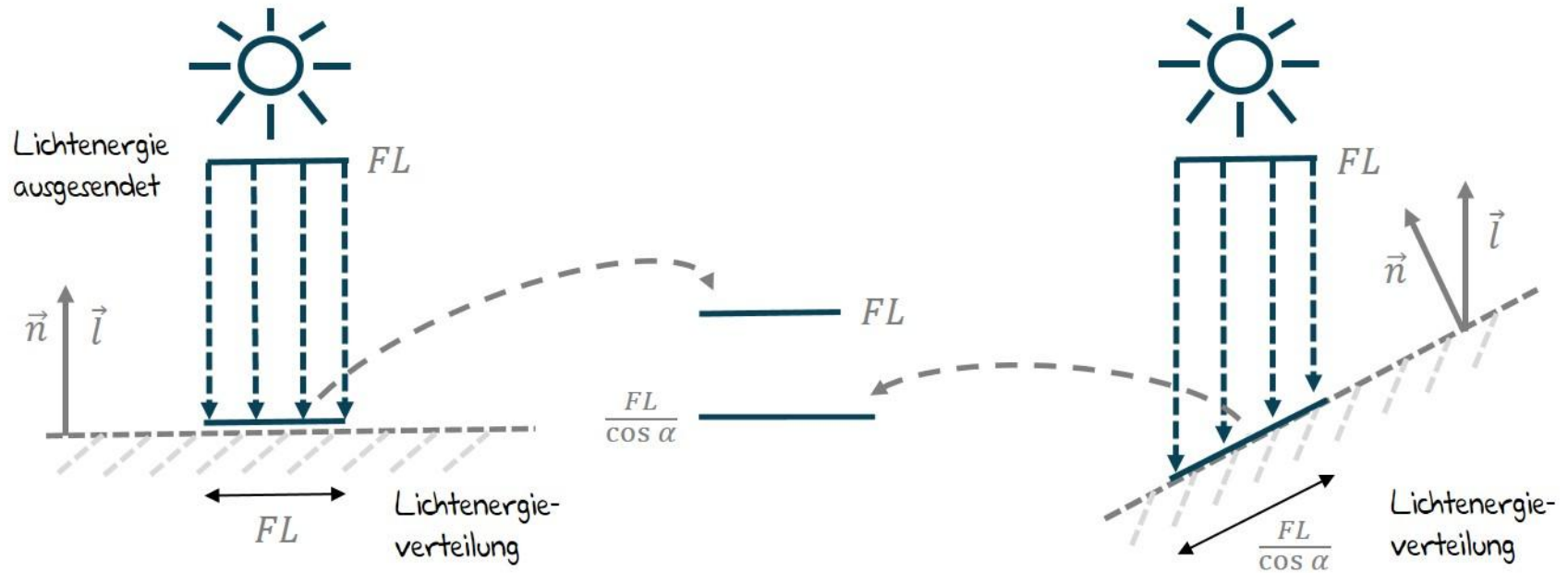
Diffuse Reflexion

Der Winkel zwischen der Oberflächennormalen und der Lichtrichtung ist entscheidend. Die Position des Betrachters spielt bei der [diffusen Reflexion](#) keine Rolle.



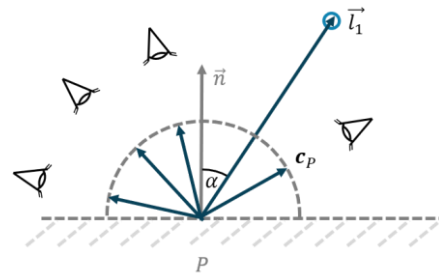
Energieverteilung und Kosinus

Je größer der Winkel beim Auftreffen auf eine Oberfläche, desto größer die resultierende Fläche ([Lambert-Kosinusgesetz](#)). Der Beobachter nimmt daher eine Abschwächung der Gesamtintensität wahr.



Diffuse Reflexion

Matte Oberflächen reflektieren einen großen Anteil diffusen Lichts. Dieser Effekt ist auch als **Lambert-Reflexion** (engl. *lambertian reflectance*) bekannt. Dabei ist der Winkel zwischen der Normalen der Objektoberfläche und der Lichttrichtung entscheidend. Die Position des Betrachters spielt dabei keine Rolle.



$$spot(\vec{n}, \vec{l}) = \max(\vec{n} \bullet \vec{l}, 0)$$

$$\mathbf{c}_P = \max(\vec{n} \bullet \vec{l}, 0) \cdot \mathbf{c}_L$$

$$\mathbf{c}_P = \mathbf{c}_P^{(d)} \cdot \max(\vec{n} \bullet \vec{l}, 0) \cdot \mathbf{c}_L$$

$$\mathbf{c}_P = \max(\vec{n} \bullet \vec{l}, 0) \cdot \mathbf{d}_P \cdot \mathbf{d}$$

$$\mathbf{c}_P = \underline{\vec{n} \bullet \vec{l}} \cdot \mathbf{d}_P \cdot \mathbf{d}$$



Diffuse Reflexion mit Richtungslicht

Vertex-Shader

```
varying vec3 color;

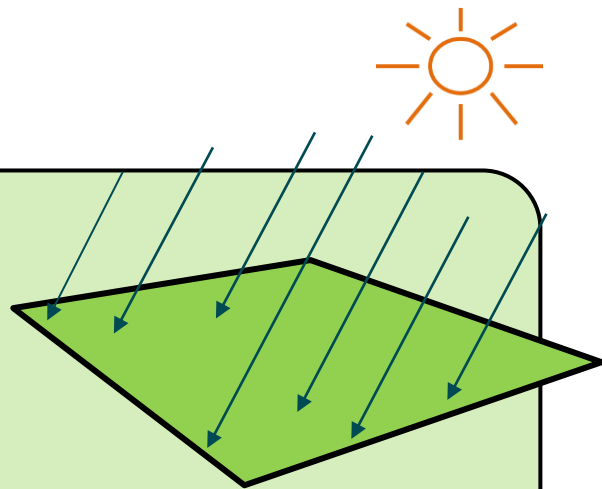
const vec3 diffuseLight    = vec3(1, 1, 1);
const vec3 lightDirection  = vec3(0,-1,-1);

uniform float diffuseWeight;

void main() {
    vec3 l = normalize(lightDirection);
    vec3 n = normalize(gl_NormalMatrix * gl_Normal);
    float diffuseLightWeight = max(dot(n, l), 0);

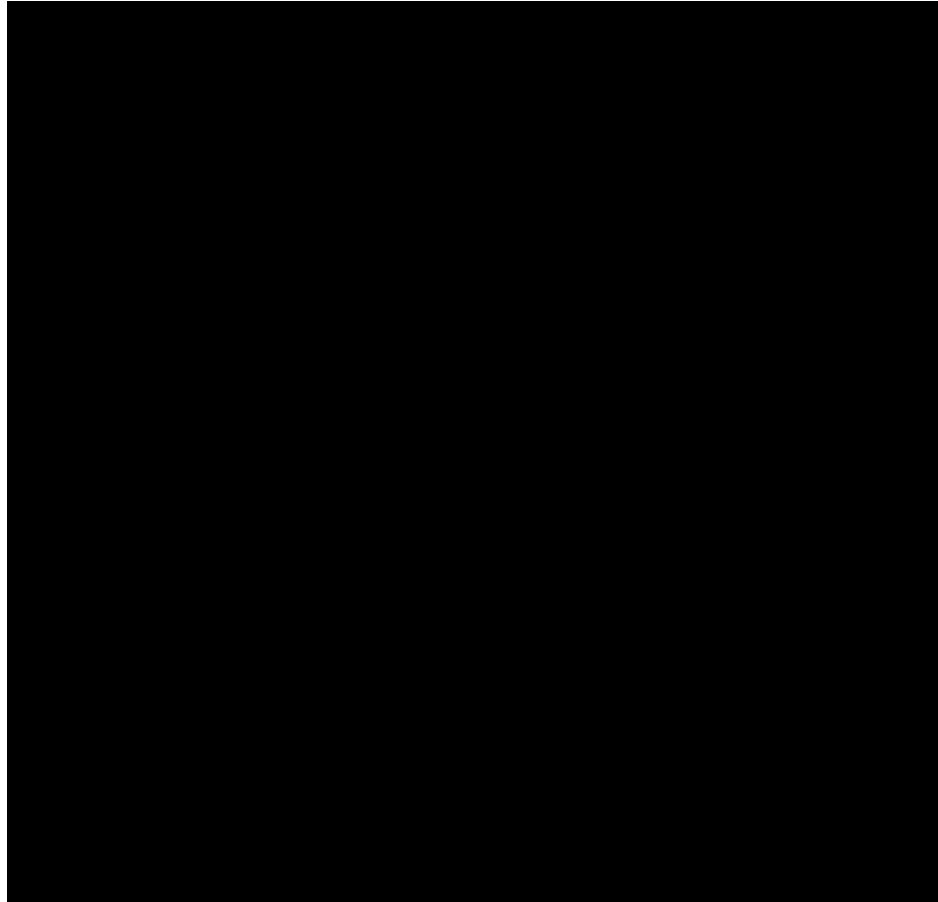
    vec3 diffuseMaterial = vec3(diffuseWeight, diffuseWeight, diffuseWeight);
    color                = diffuseLightWeight * diffuseMaterial * diffuseLight;

    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



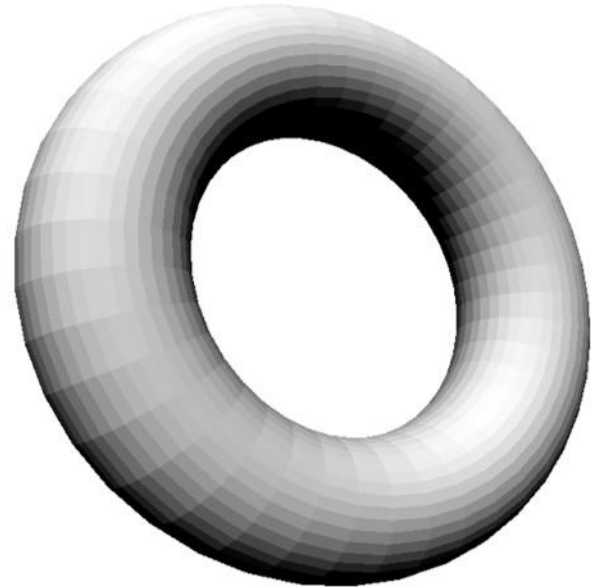
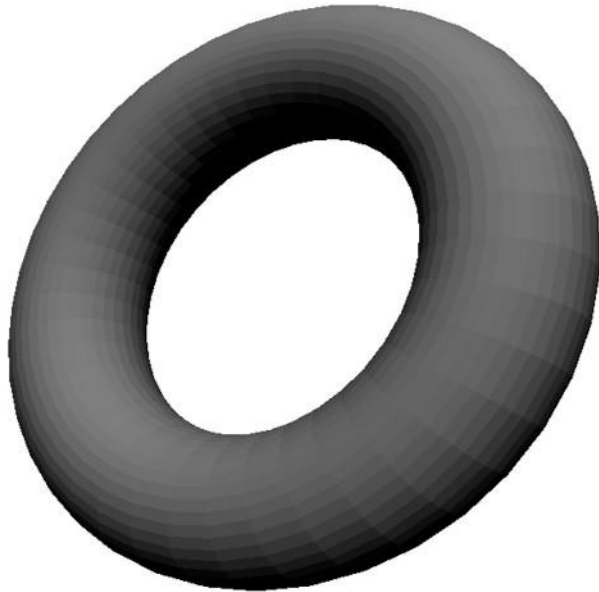
$$\mathbf{c}_P = \vec{n} \bullet \vec{l} \cdot \mathbf{d}_P \cdot \mathbf{d}$$

Diffuse Reflexion



Diffuse Reflexion

Im Beispielprogramm lässt sich der [Einfluss der diffusen Reflexion](#) prozentual angeben. Links liegt der Anteil bei 50 und rechts bei 100 Prozent.





Wie können wir aus dem Richtungslicht ein
Positionslicht machen?

Diffuse Reflexion mit Positionslicht

Vertex-Shader

```
varying vec3 color;

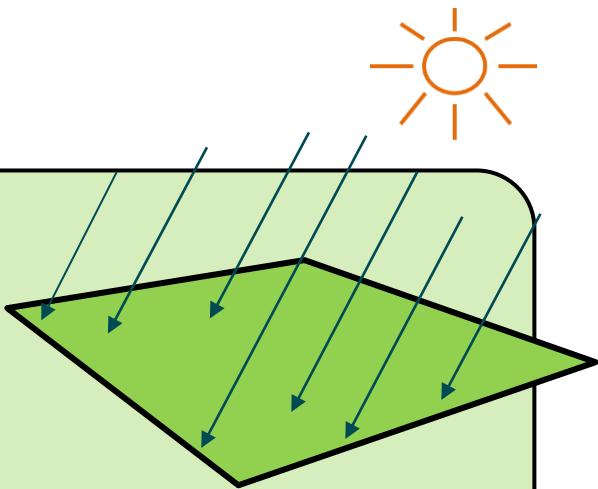
const vec3 diffuseLight    = vec3(1, 1, 1);
const vec3 lightDirection  = vec3(0,-1,-1);

uniform float diffuseWeight;

void main() {
    vec3 l = normalize(lightDirection);
    vec3 n = normalize(gl_NormalMatrix * gl_Normal);
    float diffuseLightWeight = max(dot(n, l), 0);

    vec3 diffuseMaterial = vec3(diffuseWeight, diffuseWeight, diffuseWeight);
    color                = diffuseLightWeight * diffuseMaterial * diffuseLight;

    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



Diffuse Reflexion mit Positionslicht

Vertex-Shader

```
varying vec3 color;

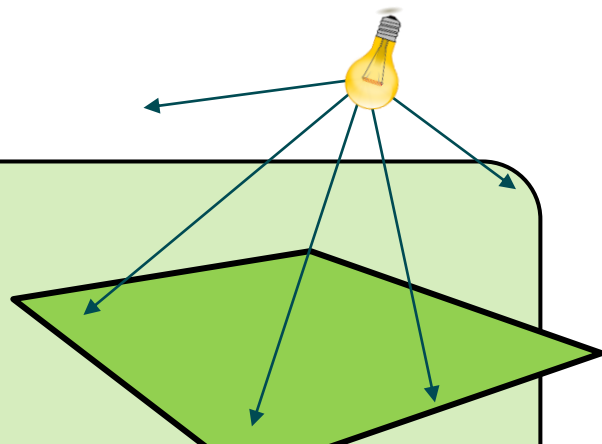
const vec3 diffuseLight    = vec3(1, 1, 1);
const vec3 lightDirection = vec3(0,-1,-1);

uniform float diffuseWeight;

void main() {
    vec3 l = normalize(lightDirection);
    vec3 n = normalize(gl_NormalMatrix * gl_Normal);
    float diffuseLightWeight = max(dot(n, l), 0);

    vec3 diffuseMaterial = vec3(diffuseWeight, diffuseWeight, diffuseWeight);
    color                = diffuseLightWeight * diffuseMaterial * diffuseLight;

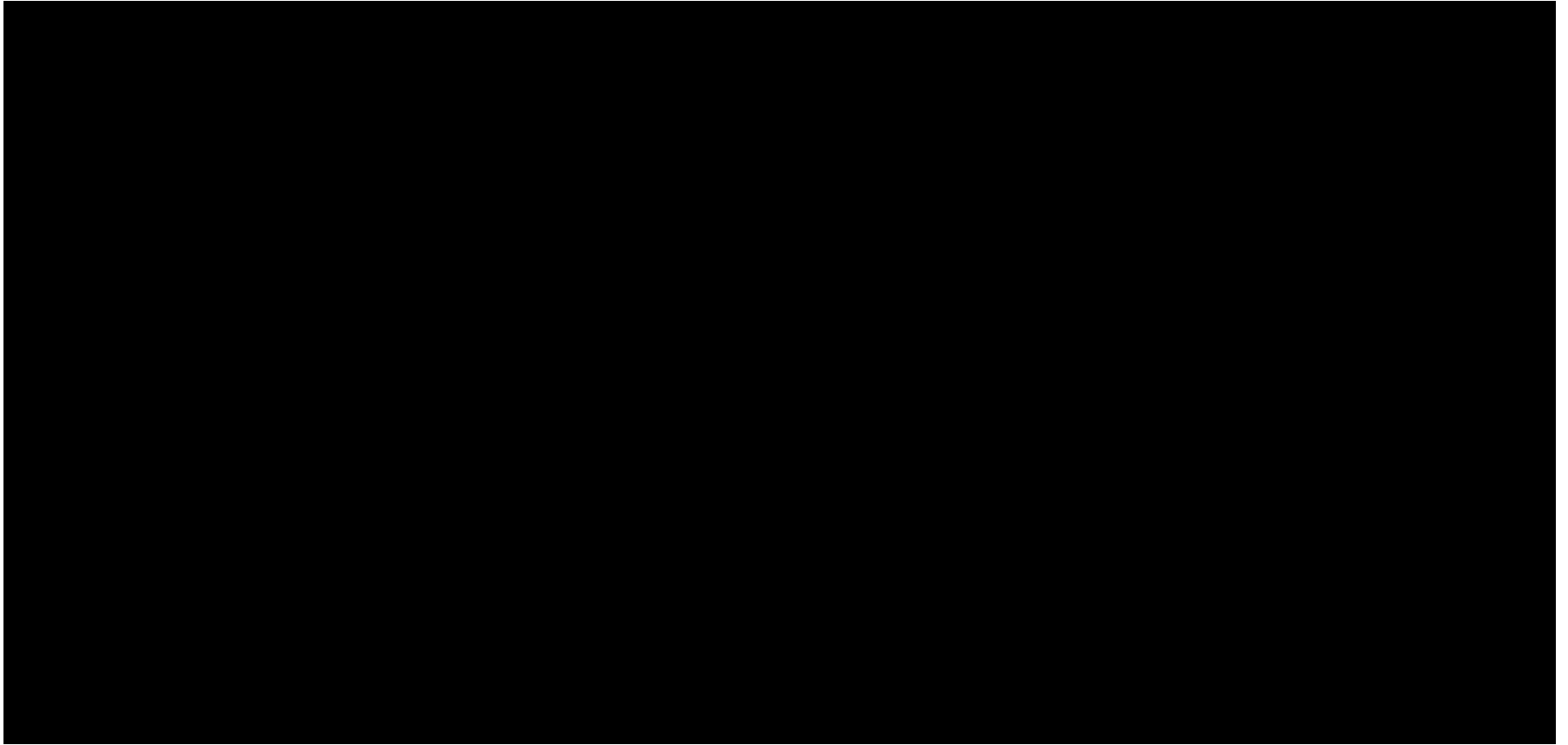
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



```
const vec3 lightPosition = vec3(0,-5,-5);
```

```
vec3 l = normalize(lightPosition - ((gl_ModelViewMatrix * gl_Vertex).xyz));
```

Diffuse Reflexion mit Positionslicht



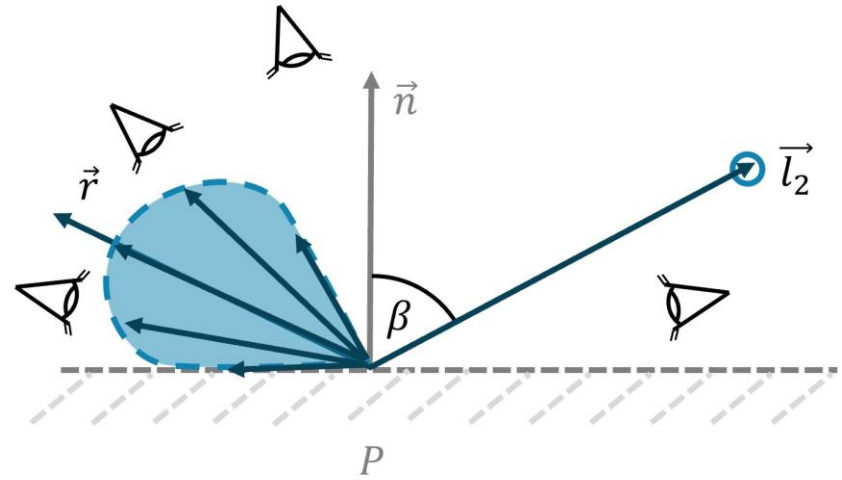
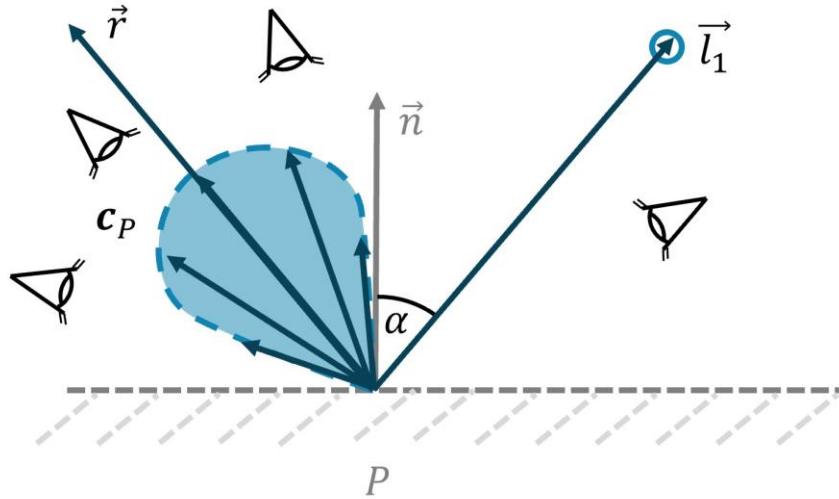


Elementares Beleuchtungsmodell

Spiegelndes Licht

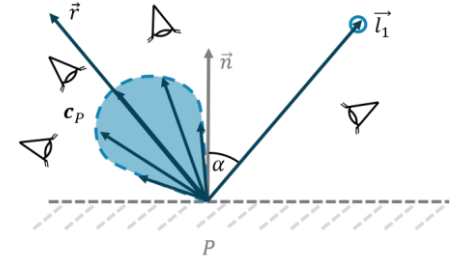
Spiegelnde Reflexion

Analog zum Lichtmodell Strahler haben wir hier eine Vorzugsleuchtrichtung, die sich kegelförmig um den Reflexionsvektor herum aufbaut. Dabei spielt die Position des Betrachters eine wichtige Rolle, denn je größer der Winkel zwischen Betrachter $\vec{v}$ und Reflexionsvektor $\vec{r}$, desto abnehmender die Wahrnehmung der Reflexionsintensität. Die Position des Betrachters beeinflusst die Wahrnehmung der [spiegelnden Reflexion](#) maßgeblich, der Winkel zwischen $\vec{n}$ und $\vec{l}$ spielt hier nur eine untergeordnete Rolle.



Spiegelnde Reflexion

Bei spiegelnden oder auch spekularen Oberflächen spielt die Position des Betrachters eine entscheidende Rolle. Trifft ein Lichtstrahl auf eine perfekt spiegelnde Oberfläche, dann wird dieser in **Richtung des Reflexionsvektors** bei gleichem Winkel zurückgeworfen.



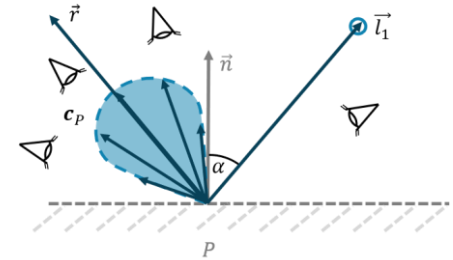
$$\mathbf{c}_P = \underline{\vec{r}} \bullet \vec{v} \cdot \mathbf{c}_L$$



Wie können wir die Lichtintensität um den
Reflexionsvektor herum konzentrieren?

Spiegelnde Reflexion

Bei spiegelnden oder auch spekularen Oberflächen spielt die Position des Betrachters eine entscheidende Rolle. Trifft ein Lichtstrahl auf eine perfekt spiegelnde Oberfläche, dann wird dieser in **Richtung des Reflexionsvektors** bei gleichem Winkel zurückgeworfen.



$$\mathbf{c}_P = \underline{\vec{r} \bullet \vec{v}} \cdot \mathbf{c}_L$$

$$\mathbf{c}_P = (\underline{\vec{r} \bullet \vec{v}})^{sp} \cdot \mathbf{c}_L$$

$$\mathbf{c}_P = (\underline{\vec{r} \bullet \vec{v}})^{sp} \cdot \mathbf{c}_P^{(s)} \cdot \mathbf{c}_L$$

$$\mathbf{c}_P = (\underline{\vec{r} \bullet \vec{v}})^{sp} \cdot \mathbf{s}_P \cdot \mathbf{s}$$





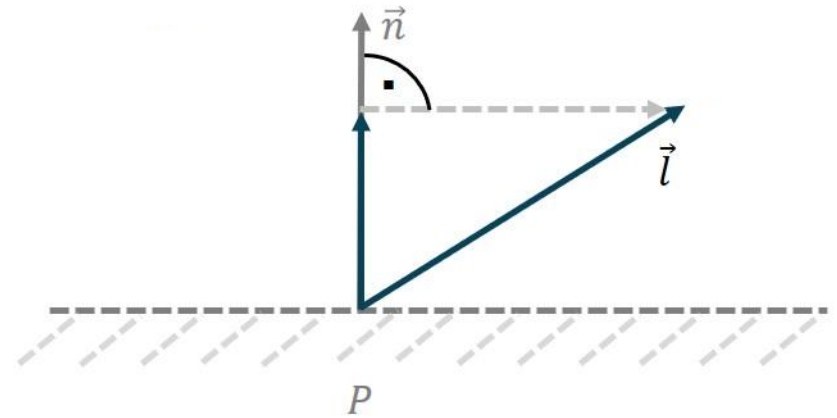
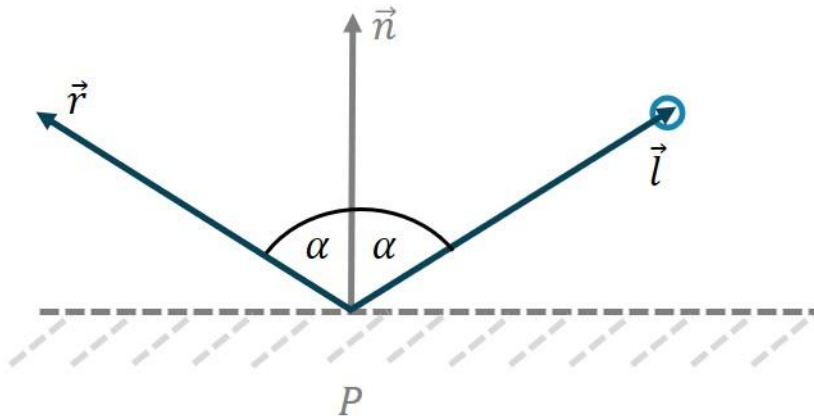
Zunächst müssen wir den
Reflexionsvektor bestimmen.

Reflexionsvektor bestimmen

Bestimmung des Reflexionsvektors mit einem einfachen Kniff.

$$\vec{r} = \text{norm} \left(2(\vec{n} \bullet \vec{l})\vec{n} - \vec{l} \right) \leftarrow$$

$$\begin{aligned}\vec{r} &= \vec{l} - 2 \cdot [\vec{l} - (\vec{n} \bullet \vec{l})\vec{n}] \\ &= \vec{l} - 2\vec{l} + 2(\vec{n} \bullet \vec{l})\vec{n} \\ &= -\vec{l} + 2(\vec{n} \bullet \vec{l})\vec{n} \\ &= 2(\vec{n} \bullet \vec{l})\vec{n} - \vec{l}\end{aligned}$$





GLSL bietet dafür die Funktion **reflect** an ...

Spekulare Reflexion

Vertex-Shader

```
varying vec3 color;

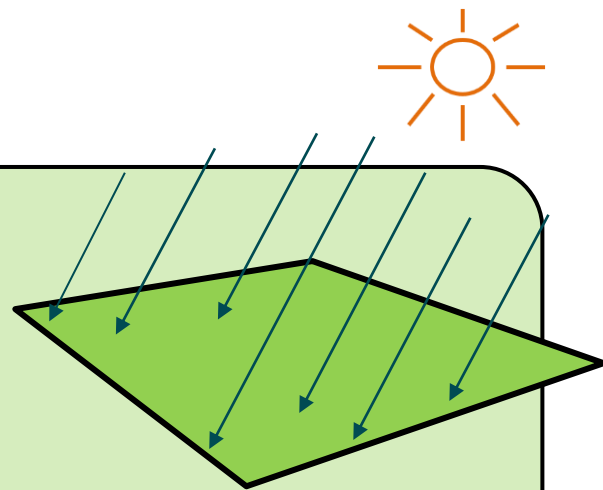
const vec3 specularLight  = vec3(1, 1, 1);
const vec3 lightDirection = vec3(0,-1,-1);

uniform float specularWeight;
uniform float specularPow;

void main() {
    vec3 l = normalize(lightDirection);
    vec3 n = normalize(gl_NormalMatrix * gl_Normal);
    vec3 v = -normalize((gl_ModelViewMatrix*gl_Vertex).xyz);
    vec3 r = reflect(-l,n);

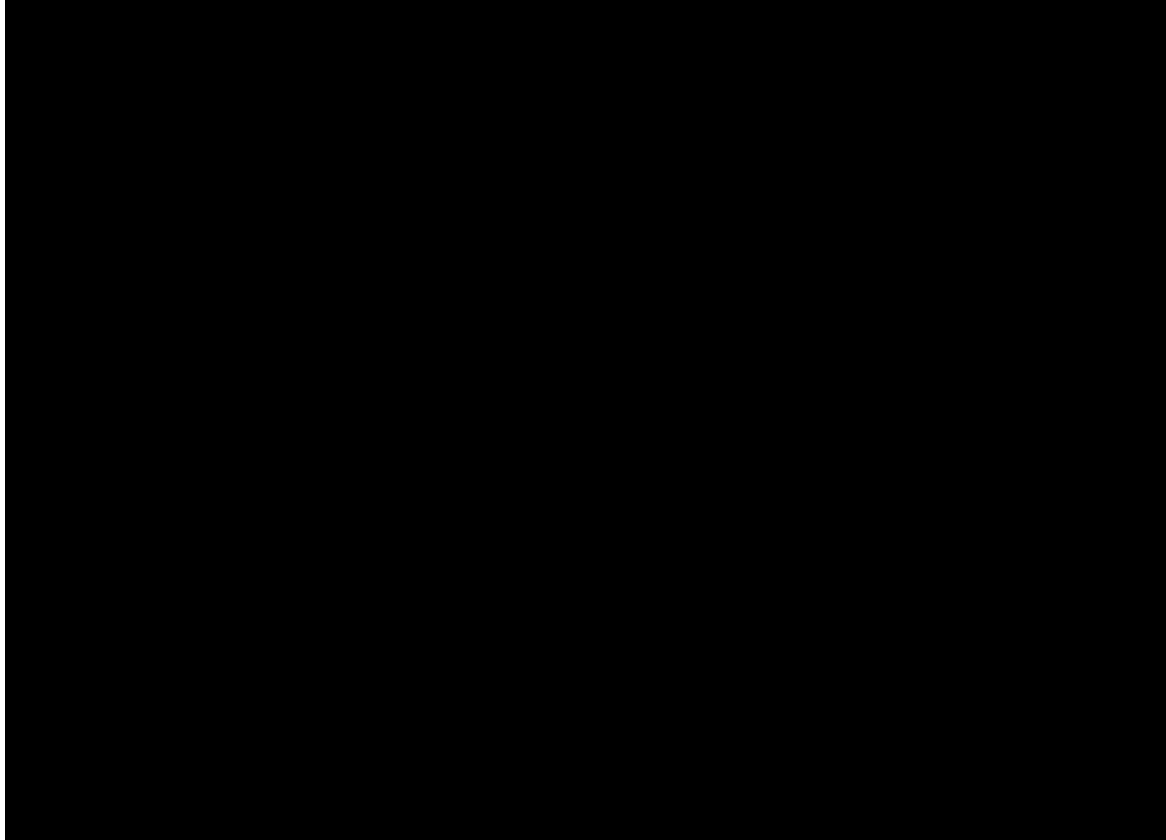
    float specularLightWeight = max(0., pow(dot(r,v), specularPow));
    vec3 specularMaterial = vec3(specularWeight, specularWeight, specularWeight);
    color = specularLightWeight * specularMaterial * specularLight;

    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot s_P \cdot s$$

Spekulare Reflexion – Richtungslicht





Auch hier können wir wieder das Richtungslicht durch ein **Positionslicht** ersetzen.

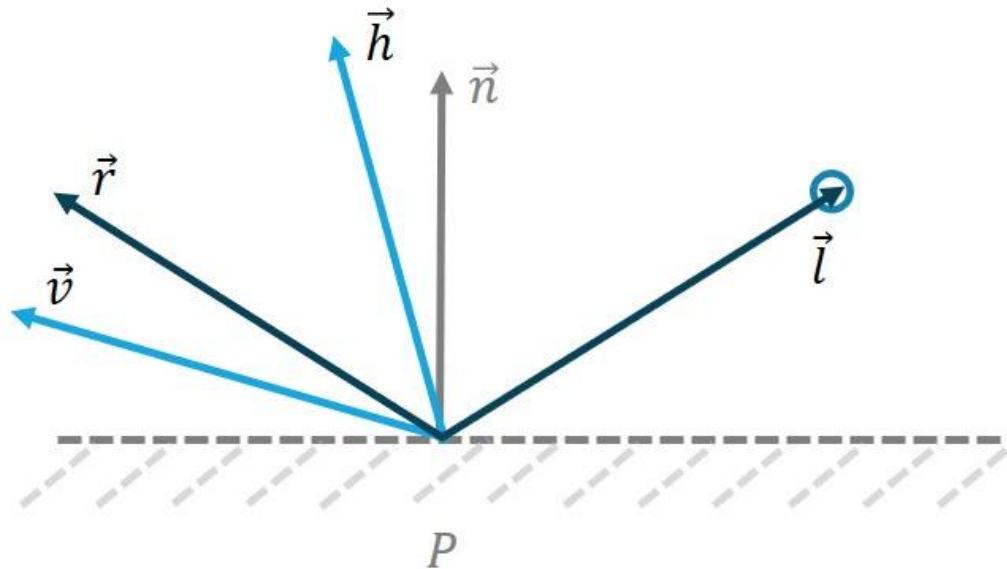
Spekular: Richtung versus Position

Richtungslicht

Positionslicht

Beschleunigung durch Halfway-Vektor

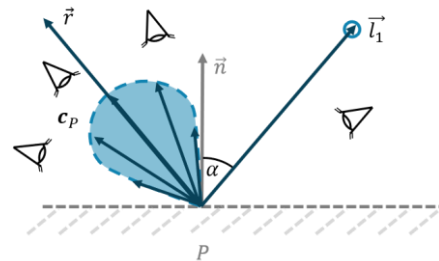
Statt des Reflexionsvektors in Abhängigkeit von $\vec{n}$ und $\vec{l}$ wird der **Halfway-Vektor** [1] als Summe von $\vec{v}$ und $\vec{l}$ bestimmt (anschließend normiert) und liegt demnach in der Mitte zwischen beiden Vektoren, denn $\vec{v}$ und $\vec{l}$ sind normiert.



[1] Blinn J.F.: Models of light reflection for computer synthesized pictures, ACM Siggraph, vol. 11, no. 2, pp.192-198, 1977

Spiegelnde Reflexion

Bei spiegelnden oder auch spekularen Oberflächen spielt die Position des Betrachters eine entscheidende Rolle. Trifft ein Lichtstrahl auf eine perfekt spiegelnde Oberfläche, dann wird dieser in Richtung des Reflexionsvektors bei gleichem Winkel zurückgeworfen.



$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot s_P \cdot s$$

$$c_P = (\vec{h} \bullet \vec{n})^{sp} \cdot s_P \cdot s$$

$$c_P = \vec{r} \bullet \vec{v} \cdot c_L$$

$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot c_L$$

$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot c_P^{(s)} \cdot c_L$$

Beschleunigung durch Halfway-Vektor

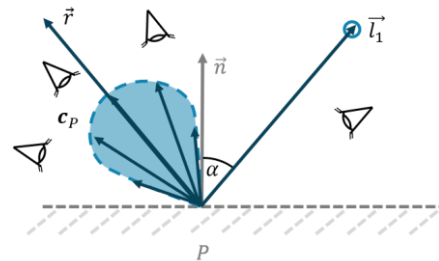


Wann lohnt sich die Berechnung überhaupt?

$$c_P = \begin{cases} (\vec{r} \bullet \vec{v})^{sp} \cdot s_P \cdot s & , \text{ wenn } \vec{n} \bullet \vec{l} > 0 \\ 0 & , \text{ sonst} \end{cases}$$

Spiegelnde Reflexion

Bei spiegelnden oder auch spekularen Oberflächen spielt die Position des Betrachters eine entscheidende Rolle. Trifft ein Lichtstrahl auf eine perfekt spiegelnde Oberfläche, dann wird dieser in Richtung des Reflexionsvektors bei gleichem Winkel zurückgeworfen.



$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot s_P \cdot s$$

$$c_P = \left(\frac{\vec{h} \bullet \vec{n}}{|\vec{h}|} \right)^{sp} \cdot s_P \cdot s$$

$$c_P = \begin{cases} \left(\frac{\vec{h} \bullet \vec{n}}{|\vec{h}|} \right)^{sp} \cdot s_P \cdot s & , \text{ wenn } \vec{n} \bullet \vec{l} > 0 \\ 0 & , \text{ sonst} \end{cases}$$



$$c_P = \vec{r} \bullet \vec{v} \cdot c_L$$

$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot c_L$$

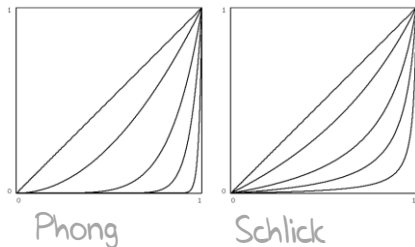
$$c_P = (\vec{r} \bullet \vec{v})^{sp} \cdot c_P^{(s)} \cdot c_L$$

Beschleunigung durch Halfway-Vektor

Beschleunigung durch Halfway-Vektor und Fallunterscheidung

Beschleunigung durch Exponentenalternative

Die Hauptrechenlast für die spekulare Reflexion liegt bei den vorgestellten Berechnungen immer noch beim Exponenten. Die Formeln selbst sind gute Annäherungen der Realität, unterliegen aber keinem physikalischen Modell. Daher können wir hier alternative Berechnungsvorschriften einsetzen, die ähnliche Eigenschaften aufweisen. Es gibt eine Vielzahl von Möglichkeiten, hier sei eine von Christophe Schlick (1994) vorgestellt.



$$t = \frac{\vec{h} \bullet \vec{n}}{\vec{h} \bullet \vec{n} + \text{sp} \cdot (1 - \vec{h} \bullet \vec{n})}$$

$$c_P = \frac{t}{\text{sp} - \text{sp} \cdot t + t}$$

$$c_P = \begin{cases} \left(\frac{t}{\text{sp} - \text{sp} \cdot t + t} \right) \cdot \text{sp} \cdot s & , \text{ wenn } \vec{n} \bullet \vec{l} > 0 \\ 0 & , \text{ sonst} \end{cases}$$

$$c_P = (\vec{r} \bullet \vec{v})^{\text{sp}} \cdot \text{sp} \cdot s$$

$$c_P = (\vec{h} \bullet \vec{n})^{\text{sp}} \cdot \text{sp} \cdot s$$

$$c_P = \begin{cases} (\vec{h} \bullet \vec{n})^{\text{sp}} \cdot \text{sp} \cdot s & , \text{ wenn } \vec{n} \bullet \vec{l} > 0 \\ 0 & , \text{ sonst} \end{cases}$$

Bisheriger Fortschritt

Beschleunigung durch Halfway-Vektor, Fallunterscheidung und Exponentenalternative



Ein elementarer Baustein fehlt allerdings noch.



Elementares Beleuchtungsmodell

Emissionslicht

Emissionslicht

Obwohl wir Objekten mit ambientem Licht bereits einen konstanten Lichtanteil zuweisen, gibt es Objekte, die selbst Licht emittieren. Sie emittieren Licht, geben also aktiv Licht ab. Allerdings hat diese Lichtabgabe in der Regel keine Auswirkung auf umliegende Objekte:

$$c_P = e \cdot e_p$$





Das waren die elementaren Bestandteile ...
... jetzt wollen wir diese kombinieren!

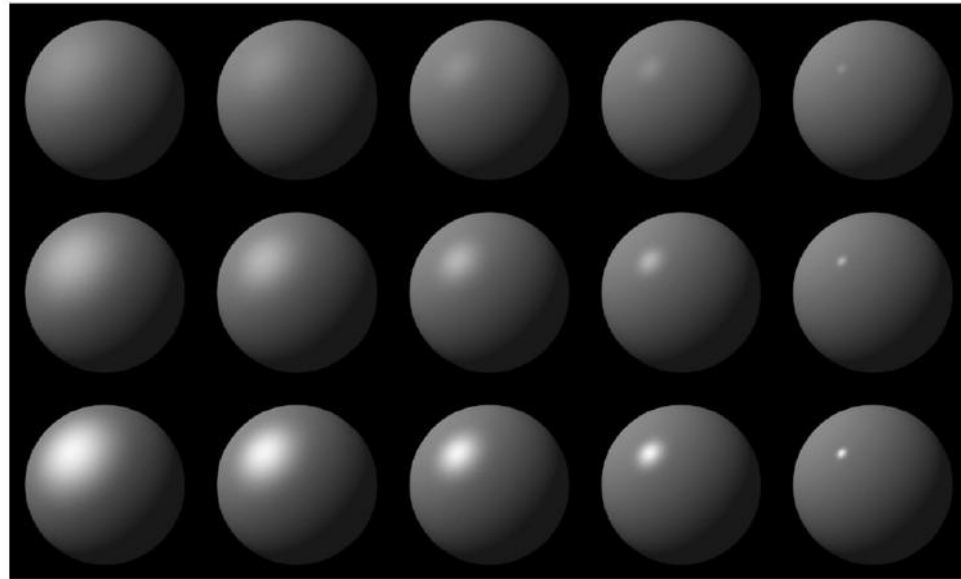
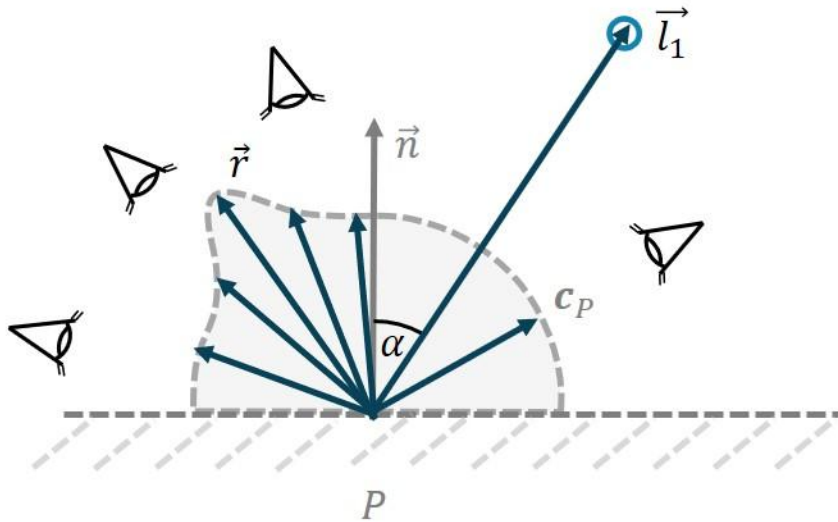


Kombiniertes Beleuchtungsmodell

Phong

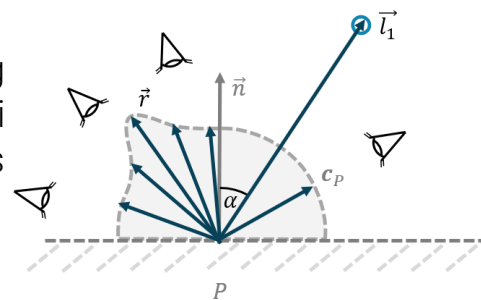
Phong-Beleuchtungsmodell

Links: Beim Beleuchtungsmodell von Phong werden der diffuse und spekulare Anteil kombiniert. Rechts: Beispiele des Phong-Beleuchtungsmodells mit unterschiedlichen Werten für den diffusen und spekularen Anteil (Abb. rechts aus [1]).



Originalartikel - Phong 1975

Ein oft verwendetes oder erweitertes Beleuchtungsmodell ist das von Bui Tuong Phong [1]. Wir kombinieren einen diffusen und einen spekularen Lichtanteil, wobei die bereits vorgestellten diffusen und spekularen Reflexionsmodelle etwas allgemeiner sind.



$$farbe = \sum_{i=1}^n [amb_{L(i)} + diff_{L(i)} + spec_{L(i)}]$$

$$c_P = \sum_{i=1}^n [a_P \cdot a_{(i)} + \vec{n} \bullet \vec{l} \cdot d_P \cdot d + (\vec{r} \bullet \vec{v})^{s_P} \cdot s_P \cdot d]$$

$$S_P = C_p [\cos(i)(1 - d) + d] + W(i) [\cos(s)]^n$$

$$farbe = diff + spec$$

$$c_P = c_L [(\vec{n} \bullet \vec{l}) \cdot (1 - d_P) + d_P] + s_P \cdot (\vec{r} \bullet \vec{v})^{s_P}$$

$$c_{L(i)} = s = d$$

Erweitertes Phong-Beleuchtungsmodell



Vertex-Shader

```
varying vec4 color;

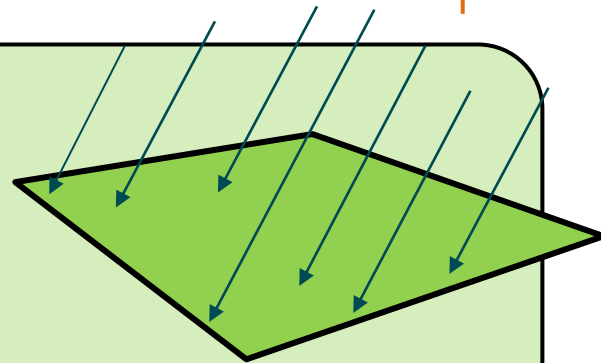
uniform float ambient;
uniform float diffuseWeight;
uniform float specularWeight;
uniform float specularPow;
uniform vec3 lightDirection;

void main() {
    vec3 l = normalize(lightDirection);
    vec3 n = normalize(gl_NormalMatrix*gl_Normal);
    vec3 v = -normalize((gl_ModelViewMatrix*gl_Vertex).xyz);
    vec3 r = reflect(-l,n);

    float specular = max(0.,pow(dot(r,v), specularPow));
    float diffuse = max(dot(n, l), 0.0);

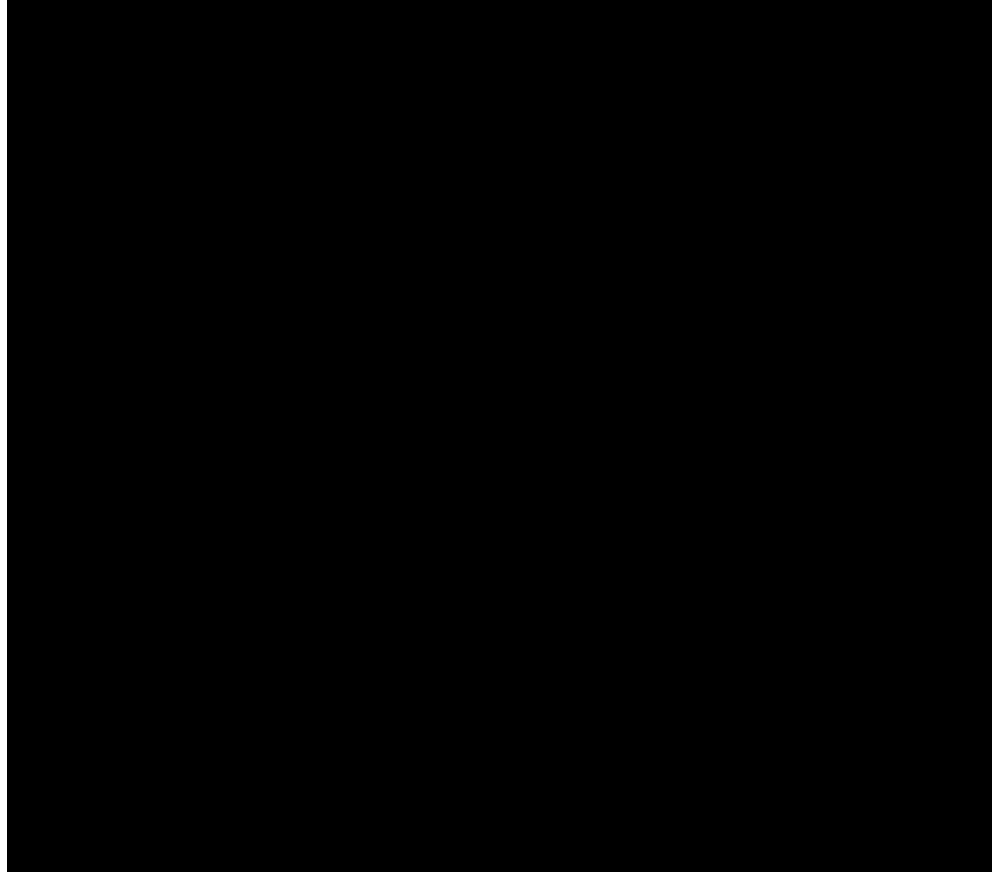
    color = vec4(1,1,1,1) * (ambient +
                             diffuse*diffuseWeight +
                             specular*specularWeight);

    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



$$c_P = \sum_{i=1}^n \left[a_P \cdot a_{(i)} + \underline{\vec{n}} \bullet \underline{\vec{l}} \cdot d_P \cdot d + (\underline{\vec{r}} \bullet \underline{\vec{v}})^{s_P} \cdot s_P \cdot d \right]$$

Erweitertes Phong-Beleuchtungsmodell





Kombiniertes Beleuchtungsmodell

OpenGL

OpenGL-Beleuchtungsmodell

Das in OpenGL verwendete Beleuchtungsmodell basiert auf dem von [Phong](#), verwendet dabei aber den Halfway-Vektor für den spekularen Anteil. So wurden weiterhin folgende Erweiterungen vorgenommen: Emissionsfarbe des Objekts, globaler Einfluss eines ambienten Lichts, Abschwächung der Intensitäten bei Positionslicht und Strahler *att*, die Strahlerfunktion *spot* zur Manipulation des Lichtkegels.

$$\mathbf{c}_{P,L(i)} = att(\vec{n}, \vec{l}_{L(i)}) \cdot spot(\vec{p}, \vec{l}_{L(i)}, \beta_{L(i)}) \cdot (\mathbf{a}_P \cdot \mathbf{a}_{L(i)} + \vec{n} \bullet \vec{l} \cdot \mathbf{d}_p \cdot \mathbf{c}_{L(i)} + \left(\vec{h} \bullet \vec{n} \right)^{sp} \cdot \mathbf{s}_p \cdot \mathbf{c}_{L(i)})$$

$$\mathbf{c}_P = \mathbf{e} \cdot \mathbf{e}_p + \mathbf{t} \cdot \mathbf{a}_P \cdot \mathbf{a}_L + \mathbf{t} \cdot \sum_{i=1}^n \mathbf{c}_{P,L(i)}$$

$$\mathbf{c}_{P,L(i)} = att(\vec{n}, \vec{l}_{L(i)}) \cdot spot(\vec{p}, \vec{l}_{L(i)}, \beta_{L(i)}) \cdot (amb_{L(i)} + diff_{L(i)} + spec_{L(i)})$$

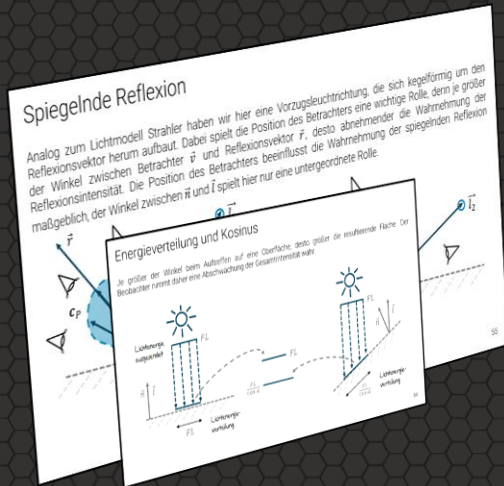
$$\mathbf{c}_P = \mathbf{e} \cdot emi + \mathbf{t} \cdot amb_L + \mathbf{t} \cdot \sum_{i=1}^n \mathbf{c}_{P,L(i)}$$

$$\mathbf{c}_{P,L(i)} = att(\vec{n}, \vec{l}_{L(i)}) \cdot spot(\vec{p}, \vec{l}_{L(i)}, \beta_{L(i)}) \cdot ([\mathbf{a}_P \cdot \mathbf{a}_{L(i)}] + [\vec{n} \bullet \vec{l} \cdot \mathbf{d}_p \cdot \mathbf{c}_{L(i)}] + [(\vec{h} \bullet \vec{n})^{sp} \cdot \mathbf{s}_p \cdot \mathbf{c}_{L(i)}])$$

$$\mathbf{c}_P = \mathbf{e} \cdot [\mathbf{e}_p] + \mathbf{t} \cdot [\mathbf{a}_P \cdot \mathbf{a}_L] + \mathbf{t} \cdot \sum_{i=1}^n \mathbf{c}_{P,L(i)}$$

Elementare Beleuchtungsmodelle

Key Takeaways



- Ambientes Licht erzeugt eine konstante Grundhelligkeit
- Diffuse Reflexion basiert auf dem Lambert-Kosinusgesetz
- Spekulare Reflexion berücksichtigt die Blickrichtung
- Reflexions- und Halfway-Vektor beschreiben Glanzlichter
- Approximationen ermöglichen effizientere Berechnungen
- Emissionslicht modelliert selbstleuchtende Objekte
- Phong kombiniert die wesentlichen Beleuchtungsanteile
- Das OpenGL-Modell erweitert Phong um zusätzliche Lichtquellentypen

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Bestimmen Sie analytisch für eine Oberfläche mit $\vec{n} = (0,0,1)$ und einer gegebenen Lichtquelle $\vec{l} = (1,0.5,1)$ den Reflexionsvektor $\vec{r}$ und geben Sie Alternativen für $\vec{r}$ an, die sich schneller berechnen lassen.
- (2) Es wurde behauptet, dass sich das vorgeschlagene Beleuchtungsmodell von Bui Tuong Phong nur aus einem diffusen und einem spekularen Anteil besteht. Lesen Sie dazu den Originalartikel von 1975 und überführen Sie die dort vorgestellte Formel schrittweise in die Notation des Skripts.

Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) Erweitern Sie den Vertex-Shadercode zum spekularen Beleuchtungsmodell, der mit einem Richtungslicht arbeitet, zu einem Positionslicht.

Aufgabe 2) Es wurden alternative Verfahren zur Beschleunigung des Reflexionsvektors vorgestellt. Implementieren Sie alle Varianten direkt im Shadercode und bestimmen Sie für geeignete Szenen mit unterschiedlichen Lichtquellen anhand geeigneter Metriken (z. B. FPS) die Performanceunterschiede.



Vielen Dank für die Aufmerksamkeit